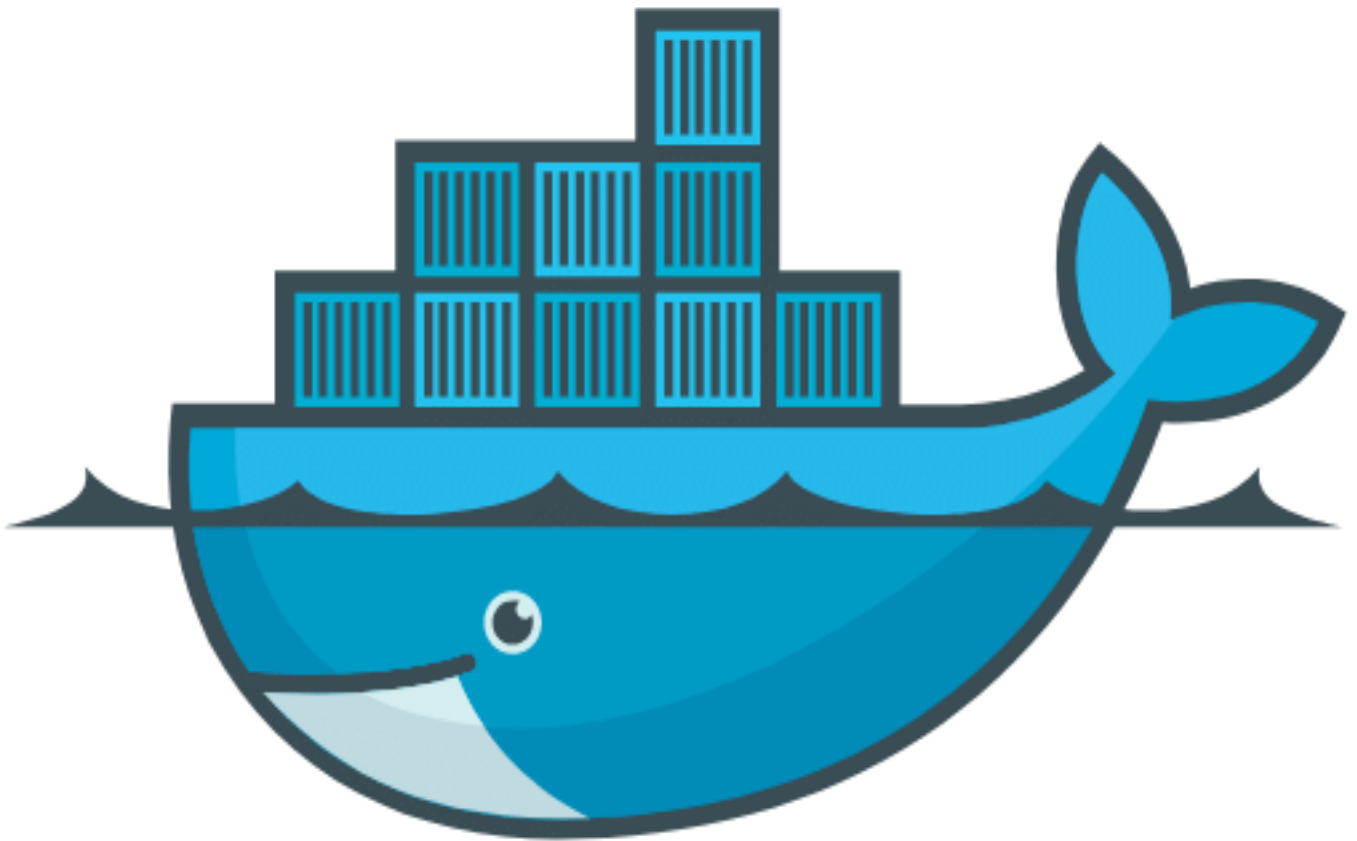


# DOCKER COMMANDS CHEATSHEET

(A COMPLETE QUICK-REFERENCE GUIDE)



## 1. Notation & Conventions

- `<angle-brackets>` placeholders you must replace (e.g., `<image:tag>`)
- `[square-brackets]` optional flags/parts
- `-d` run detached (use with `run/compose`)
- `-it` interactive TTY (use with `exec/run`)

## 2. Basics & Environment

- `docker --version` : Show Docker client version.
- `docker version` : Show client + server version.
- `docker info` : Display system-wide information.
- `docker help [cmd]` : Help for Docker or a specific command.
- `docker login [REGISTRY]` : Log in to a registry.
- `docker logout [REGISTRY]` : Log out from a registry.
- `docker context ls` : List contexts (environments).
- `docker context use <context>` : Switch current context.
- `docker system df` : Show disk usage.
- `docker system prune` : Remove unused data (use `-f` to force).
- `docker login --username <user> --password-stdin` : Secure login using stdin.

## 3. Image

- `docker image ls` : List images.
- `docker pull <image:tag>` : Pull image from registry.
- `docker push <repo/image:tag>` : Push image to registry.
- `docker build -t <name:tag> .` : Build image from Dockerfile.
- `docker image inspect <image>` : Display image details (JSON).

- `docker tag <src> <repo:tag>` : Create a new tag for an image.
- `docker rmi <image>` : Remove image(s).
- `docker image prune` : Remove dangling images.
- `docker image prune -a` : Remove all unused images (not just dangling).
- `docker history <image>` : Show history of an image.
- `docker save -o <file.tar> <image>` : Save image to tar archive.
- `docker load -i <file.tar>` : Load image from tar archive.
- `docker import` : Create image from a tarball (filesystem only).

## 4. Containers (Lifecycle)

- `docker create [opts] [cmd]` : Create a container but do not start it.
- `docker run [opts] [cmd]` : Create and start a container (e.g., `docker run --name web -d -p 80:80 nginx`).

Common `docker run` flags:

- `--rm` : Remove container automatically after exit.
- `--restart unless-stopped` : Auto-restart container.
- `-e KEY=VALUE` : Set environment variables.
- `--env-file <file>` : Load env variables from file.
- `-v <host>:<container>` : Mount volume or bind mount.
- `docker start <container>` : Start a stopped container.
- `docker stop <container>` : Gracefully stop a running container.
- `docker restart <container>` : Restart a container.
- `docker kill <container>` : Force kill a container.
- `docker pause <container>` : Pause all processes in a container.
- `docker unpause <container>` : Unpause a paused container.
- `docker rm <container>` : Remove a stopped container.
- `docker container prune` : Remove all stopped containers (use -f).

## 5. Containers (Introspection & Ops)

- `docker ps` : List running containers.
- `docker ps -a` : List all containers.
- `docker logs [-f] <container>` : Fetch container logs (follow with -f).
- `docker exec -it <container> <cmd>` : Run a command inside a running container (interactive).
- `docker attach <container>` : Attach local stdin/stdout to a container.
- `docker inspect <container>` : Inspect container details (JSON).
- `docker top <container>` : Display running processes in a container.
- `docker stats [container]` : Live resource usage statistics.
- `docker cp <container>:<path> <host_path>` : Copy files between container and host.
- `docker port <container>` : List port mappings.
- `docker rename <old> <new>` : Rename a container.
- `docker wait <container>` : Block until container stops; returns exit code.
- `docker update [opts] <container>` : Update container resource constraints (e.g., --cpus).
- `docker commit <container> <image:tag>` : Create an image from container changes.
- `docker export <container> > file.tar` : Export container filesystem to a tar.

## 6. Networking

- `docker network ls` : List networks.
- `docker network create <name>` : Create a user-defined network.
- `docker network inspect <name>` : Inspect a network.
- `docker network connect <network> <container>` : Connect a container to a network.

- `docker network disconnect <network> <container>` : Disconnect a container from a network.
- `docker network rm <name>` : Remove a network.
- `docker network prune` : Remove unused networks (use -f).
- Run options for ports & network:
  - `-p <host_port>:<container_port>` : Publish container port to host.
  - `--network <network>` : Specify network when running/building.

## 7. Volumes & Storage

- `docker volume ls` : List volumes.
- `docker volume create <name>` : Create a volume.
- `docker volume inspect <name>` : Inspect a volume.
- `docker volume rm <name>` : Remove a volume.
- `docker volume prune` : Remove unused volumes (use -f).
- Run/mount options:
  - `-v <host_path>:<container_path>` : Bind-mount or mount a volume.
  - `--mount type=bind,src=<host_path>,dst=<container_path>` : Modern mount syntax.

## 8. Build (Builder & Buildx)

- `docker build -t <name:tag> .` : Build from Dockerfile.
- `docker build --target <stage>` : Build a specific stage from a multi-stage Dockerfile.
- `docker build --build-arg KEY=VAL .` : Pass build-time arguments.
- `docker buildx create --use` : Create & switch to a Buildx builder.
- `docker buildx ls` : List builders.

- `docker buildx build [opts]` : Advanced build (multi-arch, remote, --platform, --push).
- `docker builder prune` : Prune builder cache (use -f).

## 9. Docker Compose (v2: docker compose)

- `docker compose up [-d]` : Create & start services.
- `docker compose down` : Stop & remove resources defined by Compose.
- `docker compose build [service]` : Build or rebuild services.
- `docker compose ps` : List compose services/containers.
- `docker compose logs [-f] [service]` : View logs.
- `docker compose exec <service> <cmd>` : Exec into a service container.
- `docker compose run [--rm] <service> <cmd>` : Run a one-off command.
- `docker compose restart [service...]` : Restart services.
- `docker compose stop` / `docker compose start` : Stop/start services.
- `docker compose pull` / `docker compose push` : Pull or push service images.
- `docker compose config` : Validate & view the final merged config.
- `docker compose cp <service>:<path> <host_path>` : Copy files between host and service container.
- `docker compose rm` : Remove stopped service containers.
- `docker compose ls` : List available Compose projects.
- `docker compose top` : Display running processes for services.

## 10. Swarm (Optional)

- `docker swarm init` : Initialize a Swarm.
- `docker swarm join-token worker` : Get worker join token.
- `docker node ls` : List nodes in the Swarm.

- `docker service create --name <name> -p 80:80 <image>` : Create a service.
- `docker service ls` / `docker service ps` : List services and tasks.
- `docker service update --image <img> <service>` : Update service parameters.
- `docker service scale <service>=<replicas>` : Scale service replicas.
- `docker stack deploy -c <compose.yml> <stack>` : Deploy a stack from a Compose file.
- `docker stack ls` / `docker stack ps` / `docker stack rm` : Manage stacks.
- `docker secret create <name> -` : Create a secret from STDIN.
- `docker config create <name> <file>` : Create a config object.

## 11. Security & Registry

- `docker scan <image>` : Scan image for vulnerabilities (if available).
- `docker trust` : Content trust commands (legacy, registry-managed).
- Use external `docker registry` or hosted registries for private image storage.

## 12. Troubleshooting & Cleanup

- `docker events` : Stream real-time server events.
- `Check daemon logs (host): journalctl -u docker` (or system-specific logs).
- `docker system prune -a -f` : Prune all unused images/data or removes all unused images, containers, networks, and cache.
- `docker builder prune -f` : Prune build cache.
- `docker network prune -f` / `docker volume prune -f` : Prune unused networks/volumes.